SESSION 1

Introduction to AI, Generative AI, and Agentic AI

COMPREHENSIVE STUDY NOTES

Course: Autonomous AI Systems & Agent-Based Computing

Presenter: Dinesh Wadhwani

Duration: Approximately 60 minutes

Level: Introductory

These comprehensive study notes are extracted from the full speaker script for Session 1. They contain all major concepts, detailed explanations, examples, and key discussion points. Read these notes before or after the session to deeply understand the material.

TABLE OF CONTENTS

1. COURSE OVERVIEW AND STRUCTURE

Welcome to the Course

This is the first session of a 12-session comprehensive course on Autonomous AI Systems and Agent-Based Computing. The course builds progressively from foundational concepts about artificial intelligence through to advanced topics like building multi-agent systems that can plan, reason, use external tools, and act with real autonomy.

The philosophy of this course is deeply hands-on. This is not a theoretical lecture series where you passively listen to concepts. Rather, every single session—all twelve of them—includes a lab component where you actively build something. This hands-on approach is intentional because AI is fundamentally an applied discipline. The fastest and most effective way to truly understand these concepts is not to read about them or hear about them, but to use the tools yourself, make mistakes, and iterate.

The Course's Running Project

One unique aspect of this course: you will create a fictional student's resume in today's lab, and this same resume becomes the running example throughout the entire course. This isn't busywork—it's a strategic choice. Here's how it evolves:

Session 1 (Today): You create the fictional student resume using a GenAI tool like ChatGPT. This is your first hands-on experience with generative AI.

Session 2: You return to that exact resume and test the same profile with different prompt phrasings. You'll see concretely how the wording of your prompt dramatically affects output quality.

Sessions 3-5: You refine and restructure the resume using advanced prompt engineering, structured outputs, and API-based generation.

Sessions 6-9: You compare the resume against job descriptions using embeddings, semantic search, and retrieval-augmented generation. You'll learn how to match candidates to opportunities.

Sessions 10-12: You build actual autonomous AI agents that can review, improve, tailor, and finalize the resume—all automatically. You've essentially built a hiring domain AI system.

What starts as a 5-minute warm-up lab today becomes a sophisticated, complete AI system by the end of the course. This progression is intentional: each session builds on previous ones, and you always have a concrete artifact you're improving.

Today's Session: What You'll Learn

Session 1 is foundation-building. It is intentionally concept-heavy and light on code. The goal is to build your mental model of AI—how it has evolved, what different types of AI do, and what the terminology actually means. Starting in Session 2, the material becomes progressively more hands-on and

technical. But today, use this time to ask questions and build intuition about core concepts.

By the end of today, you should be able to:

- Look at any AI product and correctly categorize what type of system it is (predictive, generative, or agentic)
- Explain the evolution of AI from its origins in the 1950s to today's agentic systems
- Understand the distinction between a chatbot, copilot, assistant, and agent
- Grasp what makes a system 'agentic' versus merely responsive
- Create a fictional student profile and generate a resume using GenAI tools

2. THE HISTORICAL EVOLUTION OF AI (1950s–PRESENT)

To understand modern AI, we need to understand where it came from. ChatGPT did not appear out of nowhere in 2022. Rather, AI has progressed through seven decades of development, including periods of intense optimism and periods of near-total stagnation (called 'AI winters'). Understanding this history helps us contextualize where we are and where we're headed.

The 1950s: Birth of AI

1950: Alan Turing's "Computing Machinery and Intelligence"

Alan Turing, a brilliant British mathematician, published a seminal paper posing a deceptively simple question: "Can machines think?" Rather than attempting to philosophically define what 'thinking' means—a question philosophers had grappled with for centuries—Turing proposed a practical test. He described a scenario where a human evaluator engages in text conversation with two entities: one human and one machine. If the evaluator cannot reliably distinguish which is which, the machine passes Turing's test and exhibits intelligence.

This was profoundly insightful. Turing suggested that intelligence should be judged by behavior and capability, not by whether the underlying mechanism mimics the human brain. While the Turing Test itself has been heavily criticized (it measures conversational ability, not true intelligence, and humans are often fooled by simple tricks), Turing's framing influenced AI research for decades.

1956: The Dartmouth Workshop

The official birthplace of AI as a formal discipline. Researchers gathered at Dartmouth College in the summer of 1956 with ambitious goals: to explore the possibility that machines could simulate human intelligence. This wasn't a small academic meeting—it brought together giants like John McCarthy (who coined the term 'Artificial Intelligence'), Marvin Minsky, Nat Rochester, and Claude Shannon. The workshop optimistically assumed that given enough computational power, machines could solve any intellectual problem.

At this point, the work was almost entirely theoretical. The computational hardware available was primitive by modern standards—computers filled entire rooms, had minimal memory, and could perform only a fraction of the calculations a modern smartphone can do. The implementation of AI would need to wait decades to catch up with the theory.

The 1960s-1970s: Early Enthusiasm and First AI Winter

Following the Dartmouth Workshop, there was significant optimism and funding. Early AI systems showed promise in narrow domains. Researchers could solve simple logic puzzles and prove mathematical theorems. Government agencies and corporations saw potential and invested heavily.

However, this optimism was premature. The computational limitations of the era became apparent quickly. Problems that seemed simple theoretically required enormous computational resources

practically. For example, exploring all possible chess moves just a few moves deep required millions of calculations. Additionally, AI systems were brittle—they worked in carefully controlled domains but failed catastrophically when facing real-world complexity and exceptions.

Funding agencies like DARPA, which had been enthusiastically supporting AI research, began to scale back support when systems failed to deliver on overpromised capabilities. By the mid-1970s, interest had largely evaporated. This period became known as the '**First AI Winter**'—a decade-long period of reduced funding, reduced research activity, and sharply diminished public interest.

The 1980s: Expert Systems and the Second AI Winter

During the 1980s, AI research took a different approach through '**Expert Systems**'. The idea was pragmatic: instead of trying to create general-purpose intelligent machines, why not focus on encoding the knowledge of human experts in specific domains?

How Expert Systems Worked: Domain experts (like a doctor, a geologist, or a mechanical engineer) would be interviewed extensively. Their decision-making logic would be converted into thousands of '**if-then rules**' and stored in a knowledge base. For example, a medical diagnosis system might contain rules like:

```
"IF patient has fever AND rash AND recently traveled to West Africa AND
lymph nodes are swollen, THEN suspect Ebola."
```

When a user provided facts about a patient, the system would match those facts against its rules to reach a diagnosis. Remarkably, these systems worked well. '**MYCIN**', an expert system for diagnosing bacterial infections, sometimes outperformed human specialists in diagnostic accuracy.

Why Expert Systems Ultimately Failed: Despite early success, expert systems proved brittle and unmaintainable:

- Brittleness: If a situation fell even slightly outside the anticipated patterns, the system had no mechanism to adapt or reason creatively.
- Maintenance nightmare: With thousands of hand-written rules, maintaining consistency and handling contradictions became impossibly complex.
- Non-scalability: Encoding expertise required hiring domain experts and spending months extracting and formalizing their knowledge. This was expensive.
- Limited adaptability: As new knowledge emerged or new situations arose, updating the rules required expensive expert consultation again.

By the late 1980s, the limitations became clear. The initial excitement faded, leading to another AI Winter lasting into the 1990s.

The 1990s-2000s: Machine Learning Renaissance

The fundamental insight that changed everything: instead of a human encoding rules, what if the system learned the rules itself from data? This shift from '**rule-based systems**' to '**statistical, data-driven learning**' marked the beginning of the Machine Learning era.

The Machine Learning Paradigm: Rather than explicitly programming every rule, machine learning systems are trained on labeled examples. For instance, consider email spam filtering. Instead of hard-coding rules like "if the email contains the word 'buy', it's spam," a machine learning system would examine thousands of emails labeled as 'spam' and 'legitimate'. It would identify statistical patterns—perhaps certain word frequencies, sender characteristics, or formatting traits—associated with spam. The system learns from these patterns and can then apply them to new, unseen emails.

Why This Was Transformative:

- Flexibility: ML systems can adapt to new variations in data without explicit reprogramming.
- Scalability: Systems don't require expensive domain experts to hand-code rules.
- Improvement over time: As new data becomes available, the system can be retrained to incorporate new patterns.
- Robustness: Statistical patterns capture real-world complexity better than hard-coded rules.

Machine learning proved vastly more powerful and practical than expert systems for many real-world applications. Spam filters, recommendation systems, and fraud detection systems became increasingly sophisticated and effective.

2012-2019: The Deep Learning Boom

A specific subset of machine learning called '**Deep Learning**'—based on artificial neural networks—emerged as revolutionary in the 2010s. The defining moment came in 2012. A neural network called '**AlexNet**' participated in ImageNet, a large-scale image recognition competition that was the benchmark for computer vision systems. AlexNet's error rate—the percentage of images it misclassified—was nearly **half** that of the previous best non-neural-network approaches. This dramatic performance gap shocked the field.

Why Deep Learning Took Off Now (Not Before): Neural networks had been theorized and researched since the 1950s and 1960s. Yet it took until 2012 for them to dominate. What changed? Two technological developments:

1. Big Data: The internet had generated unprecedented quantities of labeled data. ImageNet itself contained 14 million labeled images. Training data at this scale was unimaginable in the 1980s.

2. GPU Computing: Graphics Processing Units, originally designed for rendering video game graphics, proved exceptionally good at the matrix mathematics that neural networks require. This hardware acceleration made training large neural networks computationally feasible. Training AlexNet on a GPU took weeks; on a CPU it would have taken months or years.

What Deep Learning Could Do: Deep learning systems didn't just classify data (assigning a label to an input); they discovered hierarchical patterns. In computer vision, lower layers in a neural network might learn to detect simple features like edges, middle layers learn to detect shapes like corners and curves, and deeper layers learn to detect complex concepts like 'faces', 'dogs', or 'cats'. This hierarchical learning—mimicking how the human visual system works—was far more powerful than previous machine learning approaches.

Deep learning breakthroughs cascaded: improved image recognition (with applications from medical imaging to autonomous driving), speech recognition (enabling voice assistants like Siri), and natural language understanding. By 2019, deep learning had become the dominant paradigm in AI.

2020-2022: Generative AI Emerges

The next major shift came when deep learning models were scaled to unprecedented size and trained on enormous amounts of text data. Models like GPT-3 (released by OpenAI in June 2020) and image generators like DALL-E (2021) demonstrated something remarkable: these models could not just classify or predict—they could generate entirely new, original content.

What They Could Do: An AI could write an essay on a topic it's never been trained specifically on, write functional code in multiple programming languages, generate poetry, create images from text descriptions, translate between languages, summarize documents, and much more.

Why This Mattered: Previous AI systems were useful for specific, narrow tasks (predicting tomorrow's weather, filtering spam, recognizing faces). They operated passively within defined domains. Generative AI, by contrast, could be applied to an almost unlimited range of creative and intellectual tasks. This broader applicability is why generative AI caught the public imagination in a way previous AI advances had not.

The turning point came with the release of ChatGPT in November 2022. It achieved 100 million users faster than any other application in history. Suddenly, non-technical people had hands-on experience with AI. The field transitioned from academic curiosity to mainstream technology.

2023-Present: Agentic AI Emerges

The most recent shift—and the reason this course exists—is the emergence of '**Agentic AI**'. Generative AI models are, in many ways, reactive: they respond to prompts but don't take initiative or operate over multiple steps with planning and memory. Agentic systems are fundamentally different.

Agentic systems can:
- Plan sequences of actions toward a goal
- Make decisions about which tools to use
- Observe the results of their actions
- Adjust their approach based on feedback
- Maintain memory across multiple interactions
- Operate with minimal human direction

This represents a fundamental shift from AI that assists humans (responding to their queries) to AI that can take autonomous action toward goals. We'll explore agentic systems in depth throughout this course, but the key insight is that agentic AI operates **in pursuit of goals**, not just **in response to prompts**.

3. THE EVOLUTION LADDER: FIVE LEVELS OF AI DEVELOPMENT

Understanding this evolution ladder is the most important takeaway from today's session. If you remember only one thing from Session 1, let it be this framework. This ladder organizes all the AI history we just covered into five distinct levels, each building on and encompassing the capabilities of the levels below it.

LEVEL 1: TRADITIONAL AI (Rule-Based, Expert Systems)

Core Mechanism: Humans write explicit rules. If-then logic trees encode domain expertise.
Characteristic: Deterministic—the same input always produces the same output.
Adaptability: Low. The system has no way to handle situations outside its programmed rules.
Examples: Expert systems for medical diagnosis, rules-based chess programs, early autocorrect systems.

Traditional AI is brittle. When facing unexpected inputs or contexts, it fails catastrophically. For instance, a medical diagnosis system might have a rule: "IF fever > 38°C AND has rash THEN suspect measles." But what if the fever is 37.9°C? The system has no mechanism to handle near-misses—the rule doesn't fire, and no diagnosis is suggested.

LEVEL 2: MACHINE LEARNING

Core Mechanism: Systems learn patterns from data rather than having rules explicitly programmed.
Characteristic: Statistical—the system discovers associations and probabilities from training data.
Adaptability: Higher than traditional AI. The system can handle variations and new cases similar to training data.
Examples: Spam filters, recommendation systems, predictive analytics, fraud detection.

In machine learning, instead of a doctor manually defining rules, the system is trained on thousands of patient records labeled with correct diagnoses. It learns statistical patterns: maybe patients with high fever, rash, and certain antibody levels tend to have measles. When it encounters a new patient with similar patterns, it applies what it learned. The key advantage: it can handle edge cases and variations far better than rule-based systems.

LEVEL 3: DEEP LEARNING

Core Mechanism: Neural networks with many layers automatically discover hierarchical features.
Characteristic: Learns multi-level abstractions—can recognize complex patterns by combining simple features.
Adaptability: Very high on complex tasks. Dramatically improved performance on vision, speech, and language tasks.
Examples: Image recognition, speech recognition, machine translation, early language models.

The breakthrough: instead of humans engineering features, deep neural networks discover features automatically. In computer vision, lower layers might discover edges and colors, middle layers discover shapes like circles and lines, and deeper layers discover objects like faces and cars. This hierarchical discovery of abstraction is powerful and much more effective than previous approaches.

LEVEL 4: GENERATIVE AI (Large Language Models)

Core Mechanism: Massive neural networks trained on enormous text data, designed to generate new text.
Characteristic: Creative—can generate novel, original content that doesn't exist in training data.
Adaptability: Extremely high. Can be applied to almost any language task via prompting.
Examples: ChatGPT, GPT-4, Claude, Gemini, text-to-image models like DALL-E.

Generative AI doesn't just analyze and classify data; it creates new data. Given a prompt like "Write a poem about quantum computing," it produces original poetry. Given "Write a Python function to sort a list," it writes functional code. This creative capability represents a qualitative leap from previous AI systems.

LEVEL 5: AGENTIC AI

Core Mechanism: Autonomous systems that plan multi-step actions, use tools, remember context, and learn from feedback.
Characteristic: Goal-directed—pursues objectives with planning and reasoning, not just responding to prompts.
Adaptability: Highest. Can handle novel situations by combining reasoning, tool use, and learning.
Examples: Autonomous agents for project management, research, coding, recruiting.

Agentic AI represents the frontier. These systems don't wait for a prompt; they take initiative. A recruiting agent might autonomously review applications, schedule interviews, check references, and make recommendations. A research agent might identify gaps in knowledge, search databases, read papers, and synthesize findings. The key distinction: agentic systems pursue goals, not just respond to inputs.

Important Note on This Ladder: This is not a rigid taxonomy. There's overlap between categories. Real AI systems often combine elements from multiple levels. For instance, a modern recommendation system might use machine learning to predict user preferences (Level 2), deep learning for feature extraction (Level 3), and generative AI to create personalized text recommendations (Level 4). Additionally, systems don't make sharp jumps—there's a spectrum of increasing capability.

4. THREE TYPES OF AI: PREDICTIVE, GENERATIVE, AGENTIC

While the 'Evolution Ladder' describes how AI systems have developed over time, this section describes types by **function**—what the AI does. These are orthogonal concepts: a system can be at any level on the evolution ladder and serve any of these three functions. Understanding these three functional types helps you evaluate any AI system you encounter.

TYPE 1: PREDICTIVE AI — "What will happen?"

Core Question: Given data about the past, what will happen in the future or what category does this belong to?
Function: Analyzes historical data to forecast future outcomes or classify new data based on learned patterns.
Mechanism: Learns from labeled examples where outcomes are known, then applies that learning to new cases.

Real-World Examples:

• **Weather Forecasting:** Models analyze decades of historical weather data and current conditions to predict tomorrow's temperature, precipitation, and wind patterns.

• **Stock Price Prediction:** Systems analyze market trends, company fundamentals, economic indicators, and historical patterns to predict future stock prices.

• **Disease Diagnosis:** AI analyzes patient symptoms, medical history, test results, and patterns from thousands of similar cases to predict the likelihood of specific diseases.

• **Recommendation Engines:** Systems analyze your purchase history and preferences of similar customers to predict which products you'll like and buy them.

• **Churn Prediction:** Models predict which customers are likely to leave a service, enabling companies to intervene with offers or support.

• **Credit Risk:** Banks use predictive AI to score loan applications, predicting the likelihood of default.

Key Characteristic: Predictive AI looks backward (at historical data) to make predictions about a future state. It doesn't create anything new; it extrapolates patterns it has learned. The output is typically a number (a prediction) or a label (a category).

TYPE 2: GENERATIVE AI — "What can we create?"

Core Question: Given a prompt or partial input, what new content can we generate?
Function: Creates new content from scratch or transforms existing content. Not just classifying or predicting; actively producing novel outputs.
Mechanism: Trained on vast amounts of existing content, learns patterns and distributions, then samples from those distributions to generate new content.

Real-World Examples:

- **ChatGPT and Language Models:** Generate human-like text—essays, poetry, code, emails, stories—in response to prompts.
- **DALL-E and Image Generators:** Generate images from text descriptions. 'A red dragon with blue eyes in a magical forest' produces novel artwork.
- **GitHub Copilot:** Generates code completions and full functions based on comments and context.
- **Music Generation:** Models like Jukebox generate original music in specified styles and genres.
- **Video Generation:** Emerging models generate video sequences from text or images.
- **Text-to-SQL:** Models generate SQL queries from natural language descriptions of data.

Key Characteristic: Generative AI creates **novel outputs**. The output isn't a label or a number; it's new content that didn't exist before. This represents a qualitative shift in what AI can do. Most media coverage of AI in recent years focuses on generative AI because the outputs are tangible and understandable to non-experts.

TYPE 3: AGENTIC AI — "What should we do?"

Core Question: Given a goal, what actions should we take, in what sequence?
Function: Makes decisions, takes actions, and iteratively works toward goals. Plans sequences of steps, uses tools and resources, learns from outcomes, adjusts behavior.
Mechanism: Combines planning (deciding what to do), reasoning (understanding why), tool use (accessing external resources), and learning (improving from feedback).

Real-World Examples:

- **Project Management Agent:** Autonomously decomposes a large project into subtasks, assigns them to team members, tracks progress, identifies blockers, and adjusts plans.
- **Recruiting Agent:** Reviews job applications, screens candidates based on qualifications, coordinates interviews, checks references, and makes recommendations.
- **Research Agent:** Searches academic databases, reads papers, synthesizes findings, identifies knowledge gaps, and produces comprehensive reports.
- **System Optimization Agent:** Monitors system performance, identifies bottlenecks, tests hypotheses to fix them, and implements solutions autonomously.
- **Content Creation Agent:** Plans content strategy, researches topics, writes multiple versions, edits based on performance data, and publishes.
- **Autonomous Coding Agent:** Reads a bug report, locates relevant code, writes and tests a fix, and opens a pull request.

Key Characteristic: Agentic systems are **autonomous** and **goal-oriented**. They don't wait for a prompt; they take initiative. They make decisions, use resources (tools), and learn from outcomes. They're comfortable with multi-step, complex tasks that require planning and reasoning. This is the frontier of AI development and will be the focus of Sessions 8-12.

Why These Distinctions Matter:

Understanding these three types helps you evaluate an AI system's capabilities and limitations. They represent increasing levels of autonomy, complexity, and impact on the world. They also affect safety considerations—agentic systems that can take autonomous actions carry different risks than predictive systems that just make recommendations. Throughout this course, as we build systems, we'll use these categories to clarify what we're building and why.

5. UNDERSTANDING LARGE LANGUAGE MODELS (GENERATIVE AI)

Large Language Models (LLMs) are the technological foundation of modern generative AI and the starting point for building agentic systems. Today we'll cover the basics conceptually. We'll go much deeper in Session 2, but here's what you need to understand now.

What is a Language Model?

A language model is a statistical system trained to predict the next word (or 'token') in a sequence of text. That's the core idea—incredibly simple, but powerful. It learns this prediction capability from massive amounts of text data: billions of words from the internet, books, academic papers, and other text sources.

How Prediction Leads to Understanding: Here's the key insight: by practicing predicting billions of word sequences, the model implicitly learns about grammar, facts, logic, reasoning patterns, and cultural knowledge. For instance, if the model sees the prompt "2 + 2 =" thousands of times in training data, always followed by the completion "4", it learns the association. This learning happens not through explicit programming but through statistical patterns in the training data.

If you give ChatGPT a prompt like "Who was the first president of the United States?", it doesn't look up the answer in a database. Rather, it has learned from the patterns in its training data that when this question appears, the answer "George Washington" typically follows. The model is essentially completing a sentence based on what it learned.

The 'Large' Part: Parameters and Scale

'Large' in LLM refers to two things: the amount of training data (on the order of hundreds of billions or trillions of words) and the number of parameters (weights) in the neural network. Parameters are the knobs and dials the model has learned to adjust. GPT-3 has 175 billion parameters. GPT-4's exact size is proprietary but estimated to be significantly larger. These massive parameter counts allow the model to capture and represent the nuances, complexities, and subtleties of language and knowledge.

To put scale in perspective: AlexNet (the 2012 breakthrough in deep learning) had about 60 million parameters. Modern LLMs are 1000x larger. This scale enables capabilities that smaller models can't achieve.

Tokens: The Unit of Processing

LLMs don't process text character-by-character or even word-by-word. Instead, they break text into chunks called '**tokens**', roughly equivalent to words or common subwords.

Tokenization Examples:
"Hello world" → ['Hello', ' world'] (2 tokens)
"unbelievable" → ['un', 'believ', 'able'] (3 tokens)
"123" → ['123'] (1 token)
"🚀" → [rocket emoji token] (1 token)

Why Tokens Matter:

• **Cost:** Most LLM APIs charge per token. If an API costs $0.002 per token, a 1000-word essay at ~1200 tokens costs about $2.40 in input costs, plus more for output.

• **Context Window Limits:** Models have maximum token limits. GPT-4 can handle up to 128,000 tokens. If your input exceeds this, you'll get an error.

• **Efficiency:** Common words often tokenize into 1 token, while rare or technical words might be 5+ tokens. Using simpler vocabulary can reduce token count and cost.

Context Window: How Much Text Can the Model See?

The context window is the maximum amount of text (measured in tokens) that an LLM can process at once. This is a hard technical limit that fundamentally affects how you can use the model.

Context Window Sizes (as of 2024):
• GPT-3.5-turbo: 4K tokens (~3,000 words)
• GPT-4: 128K tokens (~96,000 words)
• Claude 3: 200K tokens (~150,000 words)

If your entire input (including your prompt, system instructions, and any context) exceeds the context window, the model literally cannot process it. This is why RAG (Retrieval-Augmented Generation)—which we'll cover in depth in Session 5—matters: it lets you reference far more information than fits in the context window by strategically retrieving only the most relevant information.

6. WHAT MAKES A SYSTEM AGENTIC?

In Session 3, we briefly distinguish between a reactive chatbot and an autonomous agent. This distinction is crucial because it explains why agentic AI represents a fundamentally new capability, not just a bigger or better chatbot.

Reactive vs Autonomous

Reactive Systems (Chatbots, Assistants): Wait for a user input (a prompt), then respond. The flow is: user initiates → AI responds. Even if the assistant remembers previous messages in a conversation, it's still fundamentally reactive—it doesn't initiate action.

Example: You ask ChatGPT "What's the capital of France?" It responds "Paris." You ask "Who was the president in 1850?" It responds with the answer. Each interaction is independent and initiated by the user.

Autonomous Systems (Agents): Work toward goals independently. They plan sequences of actions, decide what to do next, use tools as needed, and adjust their approach based on outcomes. They're goal-oriented, not prompt-oriented.

Example: You tell an autonomous agent "Organize a team lunch." The agent might autonomously: search restaurant databases, check team members' dietary preferences (from a file), identify restaurants matching those preferences, check availability on the team calendar, suggest times when everyone is free, book a reservation, and send a calendar invite—all without waiting for you to ask each step.

The Four Essential Capabilities of Agentic Systems

Not all autonomous systems are created equal. A truly agentic system has four essential capabilities:

1. Planning

Can break down a complex goal into subtasks and sequence them. If the goal is "Hire a new engineer," an agentic system might plan: (1) Post the job, (2) Review applications, (3) Schedule interviews, (4) Conduct interviews, (5) Check references, (6) Make offer. It understands dependencies (can't offer before interviews) and sequences accordingly.

2. Tool Use

Can decide which external tools or resources to use for each subtask. Recruiting example: use a job board to post the job, use email to contact candidates, use a video conferencing tool for interviews, use LinkedIn to verify credentials. An agentic system knows what tools are available and when to use each one.

3. Reasoning and Adaptation

Can reason about outcomes and adapt when something doesn't work. If the job board API is down, the agent doesn't just fail—it reasons about alternatives (post to LinkedIn, email to contacts, etc.). If a candidate isn't available for interviews during proposed times, the agent adapts and finds new times.

4. Memory and Learning

Maintains memory across interactions and learns from experience. An agent remembers who was already interviewed, what feedback was given, what candidates declined. It can learn patterns ("candidates from university X tend to perform well") and improve decisions over time.

Why This Matters: Most AI systems today have 1 or 2 of these capabilities. ChatGPT can reason and use some tools (function calling). A scheduling bot can use calendar tools. But a truly agentic system combines all four: planning complex sequences, using diverse tools, reasoning flexibly, and learning over time. This combination is still emerging and is the focus of cutting-edge AI research.

7. KEY TERMINOLOGY: CHATBOT VS COPILOT VS ASSISTANT VS AGENT

These four terms are often used interchangeably in casual conversation, especially in marketing materials. But they have distinct meanings in AI. Understanding these distinctions is crucial for accurately evaluating what a system actually does.

CHATBOT

Definition: A system that responds to text messages or queries in a conversational interface.
Memory: Minimal or no memory between conversations. A chatbot won't remember you tomorrow. Even within a conversation, memory might be limited.
Autonomy: None. The chatbot only acts in response to user input.
Use Cases: Customer service, answering FAQs, quick information retrieval, basic assistance.
Examples: Basic customer service bots, old-school rule-based Q&A; systems, simple voice assistants.

A chatbot is reactive and stateless. Each conversation starts fresh. Think of the early versions of Siri or basic customer service bots.

COPILOT

Definition: An AI that works alongside a human in the context of their work, providing assistance and suggestions.
Memory: Aware of the human's current work context (the code they're writing, the document they're editing, etc.).
Autonomy: Minimal. Provides suggestions and completions but doesn't take independent action.
Use Cases: Assisting with complex creative or technical tasks (coding, writing, design).
Examples: GitHub Copilot (suggests code completions), Microsoft Copilot in Office, Figma's AI assistant.

A copilot understands the context of what you're currently doing and provides intelligent suggestions. GitHub Copilot, for instance, looks at the file you're working in, recent functions, and comments to suggest the next line of code. It's augmenting your capabilities, not replacing your decision-making.

ASSISTANT

Definition: A conversational AI with memory within a session. Remembers conversation history and can refer to earlier messages.
Memory: Maintains conversation history within a single session. Across different sessions, memory resets.
Autonomy: Minimal. Acts only in response to user prompts, but can handle multi-turn conversations.
Use Cases: Ongoing assistance with complex or multi-step tasks where context matters.

Examples: ChatGPT (with conversation history), Google Gemini, Claude, Bing Chat.

An assistant is the most common AI system you interact with today. It remembers what you said earlier in the conversation and can refer back to it. You can have complex, multi-turn conversations where context from earlier messages informs later responses. However, the assistant is still fundamentally reactive—it waits for your next message to continue.

AGENT

Definition: An autonomous system that makes decisions, takes actions, uses tools, and pursues goals with minimal human direction.
Memory: Long-term memory (can learn from past interactions), goal memory (knows what it's trying to achieve), and tactical memory (current task context).
Autonomy: High. Makes decisions about what to do, which tools to use, when to ask for help, and how to adjust when things don't work.
Use Cases: Complex multi-step tasks where continuous human supervision isn't practical or scalable.
Examples: AutoGPT, custom agents built with LangChain, company-specific recruiting or project management agents.

An agent is fundamentally different from the previous three. It doesn't wait for your next message. Instead, it actively works toward a goal you've set, making autonomous decisions about what to do next, what resources to use, and when to ask for human input. An agent can continue working even if you're not actively prompting it—it'll handle multi-step processes on its own.

Comparison: Where They Fall on the Autonomy Spectrum

These four aren't completely separate categories; they sit on a spectrum of increasing autonomy and capability:

Chatbot: Stateless, reactive, minimal memory.
→ **Copilot:** Context-aware, still reactive, assists human work.
→ **Assistant:** Stateful (remembers conversation), reactive, handles complex conversations.
→ **Agent:** Proactive, autonomous, goal-oriented, makes independent decisions.

8. REAL-WORLD EXAMPLES: WHERE DO THESE SYSTEMS APPEAR?

To make these categories concrete, let's look at real products you probably use and categorize them. Note that many products blend multiple categories—they're not mutually exclusive.

Spotify Recommendations

Type: Predictive AI
Why: Based on your listening history, artists you follow, and preferences of similar users, Spotify predicts which songs or playlists you'll like. It forecasts your preferences.

ChatGPT and GitHub Copilot

Type: Generative AI (creating new content)
ChatGPT category: Assistant (remembers conversation history)
Copilot category: Copilot (context-aware assistance)
Why: Both generate entirely new content—text or code—that didn't exist before, based on prompts and context.

Siri, Alexa, Google Assistant

Type: Assistant (and increasingly Copilot)
Why: These handle a range of tasks (setting reminders, playing music, answering questions, controlling smart home devices). They remember context within a session. Increasingly they're embedded in your workflow (notifications, voice commands), making them copilot-like.

Credit Card Fraud Detection

Type: Predictive AI
Why: Every time you tap your card, a model scores that specific transaction in real time for fraud risk, based on patterns like your location, transaction amount, typical spending history, and merchant type.

Autonomous Vehicles (Self-Driving Cars)

Type: Agentic AI (the most sophisticated real-world example)
Why: The car continuously perceives its environment (cameras, lidar, radar), plans a path, and acts (steering, accelerating, braking) autonomously in a live, constantly changing environment with extremely high stakes. It must reason about situations (pedestrian crossing, traffic light change) and make decisions without human direction.

Amazon Product Recommendations

Type: Predictive AI
Why: Based on your purchase history, items you've viewed, and patterns from similar customers, Amazon predicts which products you'll buy. The goal is prediction of your behavior, not recommendation of new content.

Important: Most Real Products Blend Categories

Notice something across this whole list: most real products actually blend more than one category. A modern car with adaptive cruise control uses predictive AI to estimate distance and speed of the car ahead, but it also uses agentic capabilities to automatically brake. Spotify uses predictive AI for recommendations, but its "AI DJ" feature uses generative AI to create spoken commentary.

Don't think of these categories as rigid, mutually exclusive boxes. Rather, think of them as capabilities that real products often combine. Understanding each helps you understand the whole.

9. LAB EXERCISE: CREATING YOUR FIRST GENAI RESUME

This is hands-on time, and we want everyone actively working, not just watching. This isn't a trivial warmup—the resume you create today becomes your running example throughout the entire course. You'll refine it, improve it, analyze it against job descriptions, and eventually have AI agents automatically improve it. So take it seriously.

Step 1: Invent a Fictional Student Profile

Create a fictional person. Don't overthink this—30 seconds of invention is enough. You need:

- Full name
- Degree/field of study (and what year of study they're in)
- 2-3 technical skills
- One or two internships, projects, or work experience

Example: "Priya Sharma, final-year Computer Science student, skills in Python, SQL, and basic machine learning, completed a summer internship building a data dashboard for a fintech startup."

Step 2: Open a GenAI Tool

Use any generative AI tool you have access to: ChatGPT, Claude, Gemini, Microsoft Copilot, or another. Today we're just using it—not optimizing prompts yet (that's Session 3).

Step 3: Write a Simple Prompt

Don't overthink the prompt. Something simple works fine for today. Example:

> "Write a one-page resume for a final-year computer science student named Priya Sharma with skills in Python, SQL, and machine learning, who completed a summer internship at a fintech startup building a data dashboard."

Step 4: Review What Comes Back

Does the resume look realistic? Is it well-formatted? Is it complete? Don't try to perfect it yet—rough edges are completely fine and expected. In Session 2, we'll show how changing your prompt changes the output. So seeing imperfections today is actually valuable; it sets us up for next session.

Step 5: Save It

IMPORTANT: Save both your fictional profile AND the prompt AND the resume somewhere you won't lose it (Google Docs, Notion, or locally). This isn't throwaway work. You're going to use and refine this same resume through Sessions 2-11. It becomes your running example for the entire course.

10. KEY TAKEAWAYS FROM SESSION 1

As we close today, here are the five core ideas I want everyone to remember and carry into Session 2 and beyond:

1. AI Has Evolved Through Distinct Eras. From traditional rule-based systems → machine learning → deep learning → generative AI → agentic AI. Each stage adds new capabilities; none get thrown away. Understanding this history contextualizes where we are.

2. The Evolution Ladder. Traditional AI → Machine Learning → Deep Learning → Generative AI → Agentic AI. This ladder is the single most important concept from today. If you remember nothing else, remember this framework.

3. Three Types of AI by Function: Predictive ("what will happen?") → Generative ("what can we create?") → Agentic ("what should we do?"). Real products often blend these, but understanding each helps you evaluate systems.

4. Agentic Means Autonomous, Goal-Oriented, and Tool-Using. An agent plans multi-step sequences, uses tools, reasons flexibly, and learns from outcomes. This is qualitatively different from reactive chatbots or even sophisticated assistants.

5. Chatbot, Copilot, Assistant, and Agent Aren't Marketing Fluff—They're Specific Capabilities on a Spectrum. Understanding these distinctions helps you accurately evaluate what a system actually does. Most real products blend aspects of multiple categories.

6. You've Already Generated Your First GenAI Content. That resume you built today isn't just practice. You'll iterate on it throughout the course, building it into a complete AI system. It's a concrete artifact showing the progression from simple GenAI use to sophisticated autonomous systems.

11. PREVIEW: SESSION 2 — LLM FUNDAMENTALS AND OUTPUT BEHAVIOR

In Session 1, we established conceptually what Large Language Models and Generative AI are. Next session, we open the hood and explore how LLMs actually behave under the surface—but without heavy mathematics.

Topics Covered in Session 2:

- **Tokens and Tokenization:** Why an LLM 'reads' your text the way it does, and why that matters for cost and behavior.
- **Transformers:** The architecture underneath every modern LLM, explained conceptually without deep math.
- **Pre-training vs Fine-tuning:** Why a model can know general things but still need extra training for a specific task.
- **Context Windows and Parameter Settings:** Why there's a limit to how much text you can feed a model at once, and how settings like temperature affect output randomness.
- **System Messages and Prompt Injection:** How to give models instructions, and limitations of those instructions.

Lab Exercise in Session 2:

You'll take that exact resume you built today and test it with multiple different prompt phrasings, side by side, comparing the output quality. You'll start to see, very concretely, why the exact words you choose in a prompt matter so much. This sets up prompt engineering, which we dive into properly in Sessions 3-4.

See you next time. Thank you for the questions and energy today.